

1

PYTHON – Complete Notes

Part - 1
(Page 1 of 5)

1. What is Python?

- Python is a high-level, interpreted, general-purpose programming language.
- It is easy to learn and has simple syntax.
- Created by Guido van Rossum and released in 1991.
- Python supports multiple programming paradigms (procedural, OOP, functional).

Example:

```
print("Hello, Python!")  
# Output: Hello, Python!
```

2. Variables

- Variables are used to store data.
- No need to declare the type (dynamically typed).

Example:

```
name = "Alice"      # str  
age = 21             # int  
price = 19.99        # float  
is_student = True    # bool
```

3. Data Types

- int → whole numbers
- float → decimal numbers
- str → sequence of characters
- bool → True or False
- NoneType → None

Example:

```
x = 10               # int  
y = 3.14             # float  
s = "Hello"          # str  
flag = True           # bool  
n = None              # NoneType
```

4. Operators

A. Arithmetic Operators

+	Addition
-	Subtraction
*	Multiplication
/	Division
//	Floor Division
%	Modulus
**	Exponent

B. Comparison Operators

==	Equal
!=	Not Equal
>	Greater than
<	Less than
>=	Greater or equal
<=	Less or equal

C. Logical Operators

and	→ Returns True if both are True
or	→ Returns True if one is True
not	→ Reverses the result

★ Operators are used to perform operations on variables and values.

2

PYTHON – Complete Notes

Part – 2
(Page 2 of 5)

5. Input and Output

- `input()` is used to take input from the user.
- `print()` is used to display output.

Example:

```
name = input("Enter your name: ")
print("Hello,", name)
# If user enters Alice
# Output: Hello, Alice
```

6. Type Casting

- Converting one data type to another.

Example:

```
x = int("10")           # str to int
y = float("3.14")       # str to float
z = str(100)            # int to str
print(x, y, z)          # 10 3.14 100
```

7. Conditional Statements

- Used to perform different actions based on conditions.

Example:

```
age = 18
if age >= 18:
    print("Adult")
elif age >= 13:
    print("Teenager")
else:
    print("Child")
# Output depends on age value
```

Comparison Operators

<code>==</code>	Equal
<code>!=</code>	Not Equal
<code>></code>	Greater than
<code><</code>	Less than
<code>>=</code>	Greater or equal to
<code><=</code>	Less or equal to

8. Loops

(a) For Loop

- Used to iterate over a sequence (list, tuple, string, range, etc.)

Example:

```
for i in range(1, 6):
    print(i)
# Output: 1 2 3 4 5
```

(b) While Loop

- Repeats a block of code while condition is True.

Example:

```
i = 1
while i <= 5:
    print(i)
    i = i + 1
# Output: 1 2 3 4 5
```

9. Strings

- Strings are sequences of characters.
- Enclosed in single (`' '`) or double (`" "`) quotes.

Example:

```
s = "Hello, Python!"
print(s[0])      # H
print(s[7])      # P
print(len(s))    # 13
print(s[0:5])    # Hello
```

Common String Methods

<code>s.lower()</code>	→ convert to lowercase
<code>s.upper()</code>	→ convert to uppercase
<code>s.strip()</code>	→ remove spaces
<code>s.replace(old, new)</code>	→ replace
<code>s.split()</code>	→ split into list

<code>s.startswith(x)</code>	→ True if string starts with x
<code>s.endswith(x)</code>	→ True if string ends with x
<code>s.find(x)</code>	→ find index of x (first occ.)
<code>s.count(x)</code>	→ count occurrences of x

3

PYTHON – Complete Notes

Part - 3
(Page 3 of 5)

10. Lists

- Lists are ordered, mutable collections.
- Allow duplicate values.

Example:

```
lst = [1, 2, 3, 4, 5]
print(lst[0])      # 1
print(lst[-1])     # 5
lst.append(6)       # add 6
lst.insert(2, 10)   # insert 10
lst.remove(3)       # remove 3
lst.pop()           # remove last
print(len(lst))     # length list
```

List Methods

count(x) → count occurrences of x
index(x) → return index of x

11. Tuples

- Tuples are ordered, immutable collections.
- Written using parentheses ().

Example:

```
t = (1, 2, 3, 4)
print(t[0])      # 1
print(t[-1])     # 4
print(len(t))    # 4
t[0] = 10        # Error (immutable)
```

Tuple Methods

count(x) → count occurrences of x
index(x) → return index of x

12. Sets

- Sets are unordered collections of unique items.
- No duplicate values allowed.

Example:

```
s = {1, 2, 3, 4}
s.add(5)          # {1, 2, 3, 4, 5}
s.remove(2)       # {1, 3, 4, 5}
print(3 in s)     # True
```

Set Methods

add(x) → add element
remove(x) → remove element
discard(x) → remove if exists
pop() → remove random element
clear() → remove all elements

13. Dictionaries

- Store data in key-value pairs.
- Keys must be unique and immutable.

Example:

```
d = {'name': 'John', 'age': 21, 'city': 'New York'}
print(d['name'])   # John
d['age'] = 22
d['country'] = 'USA'
print(d)
# {'name': 'John', 'age': 22, 'city': 'New York', 'country': 'USA'}
```

Dictionary Methods

keys() → all keys
values() → all values
items() → key-value pairs
get(k) → return value
update() → update dictionary
pop(k) → remove key
clear() → remove all items

14. Functions

- A function is a block of reusable code.
- Helps to organize and reuse code.

Example:

```
def add(a, b):
    return a + b
result = add(3, 4)
print(result)     # 7
```

Types of Functions

1. Built-in functions
2. User-defined functions
3. Lambda functions (anonymous)

15. Modules

- Modules are files containing Python code.
- Use 'import' to use a module.

Example:

```
import math
print(math.sqrt(16)) # 4.0
```

★ Lists, Tuples, Sets, and Dictionaries are important data structures in Python. They help store and manage data efficiently. 😊

4

PYTHON – Complete Notes

Part - 4
(Page 4 of 5)

16. Exception Handling

- Handles runtime errors gracefully.

Example:

```
try:
    x = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
else:
    print("No error")
finally:
    print("Execution completed")
```

Common Exceptions

ZeroDivisionError
ValueError
TypeError
IndexError
KeyError
FileNotFoundError

17. File Handling

- Used to work with files.

Example:

```
# Write
f = open('demo.txt', 'w')
f.write("Hello Python!")
f.close()

# Read
f = open('demo.txt', 'r')
content = f.read()
print(content)
f.close()
```

File Modes

'r' → read
'w' → write
'a' → append
'r+' → read & write
'w+' → write & read

18. Object-Oriented Programming (OOP)

- Based on classes and objects.
- Main concepts: Class, Object, Inheritance, Polymorphism, Encapsulation.

Example:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def info(self):
        return f"Name: {self.name}, Age: {self.age}"

p = Person("Alice", 20)
print(p.info()) # Name: Alice, Age: 20
```

19. Lambda Functions

- Small anonymous functions.
- Written using 'lambda' keyword.

Example:

```
add = lambda a, b: a + b
print(add(5, 3)) # 8

square = lambda x: x * x
print(square(4)) # 16
```

Use Cases:

- When function is short and used once.
- As an argument to other functions.
- With functions like filter(), map(), reduce().

20. List Comprehension

- Concise way to create lists.
- Improves readability and performance.

Example:

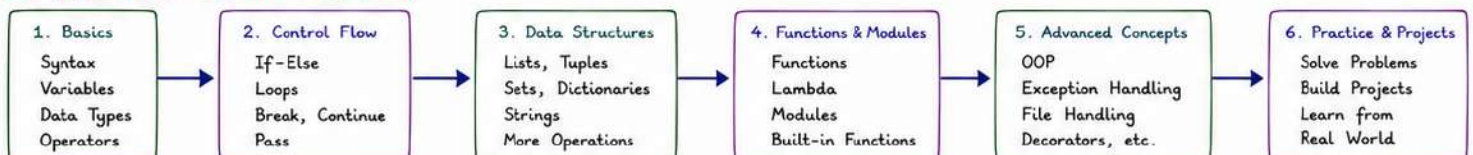
```
squares = [x*x for x in range(1, 6)]
print(squares) # [1, 4, 9, 16, 25]

even_numbers = [x for x in range(1, 11) if x % 2 == 0]
print(even_numbers) # [2, 4, 6, 8, 10]
```

Built-in Functions

filter() → Filter items
map() → Apply function
reduce() → Reduce to single value
enumerate() → Loop with index
zip() → Combine iterables

21. Python Roadmap (For Beginners)



★ Keep practicing and building projects. Consistency and curiosity will make you a great Python developer! 😊

5

PYTHON – Complete Notes

Part - 5
(Page 5 of 5)

22. Advanced Topics

(a) Tuples vs Lists

Feature	List	Tuple
Mutable	Yes	No
Syntax	[]	()
Performance	Slower	Faster
Use	When data changes	When data is constant

(b) Shallow Copy vs Deep Copy

Shallow Copy

- Copies the reference.
- Changes in original affect the copy.
- Use: `copy()`, `list()`

Deep Copy

- Copies the actual object.
- Changes in original do not affect the copy.
- Use: `copy.deepcopy()`

23. Python Interview Questions (Basic)

1. What is Python?
2. What are the key features of Python?
3. What is the difference between List and Tuple?
4. How is memory managed in Python?
5. What is the difference between `==` and `is`?
6. What are mutable and immutable objects?
7. What is a function? Types of functions?
8. What is a module? How to create a module?
9. What is exception handling?
10. What is a file? Modes of file handling?
11. What is OOP? Key principles of OOP?
12. What is inheritance? Types of inheritance?
13. What is polymorphism?
14. What is encapsulation?
15. What is lambda function?
16. What is list comprehension?
17. What are decorators in Python?
18. What is a generator?
19. What is the difference between `append()` and `extend()`?
20. What are `*args` and `**kwargs`?

24. Quick Revision – Syntax Cheat Sheet

Print : `print("Hello")`
 Input : `input("Enter")`
 If : `if condition:`
 Elif : `elif:`
 Else : `else:`
 For Loop : `for i in range(n):`
 While Loop : `while condition:`
 Def Function : `def func():`
 Return : `return value`
 Import Module : `import module_name`
 Try : `try:`
 Except : `except Exception:`
 With (File) : `with open('file.txt') as f:`

List : `[1, 2, 3]`
 Tuple : `(1, 2, 3)`
 Set : `{1, 2, 3}`
 Dictionary : `{'key': 'value'}`
 Add (List) : `lst.append(x)`
 Remove (List) : `lst.remove(x)`
 Length : `len(obj)`
 Sort : `sorted(lst)`
 Max / Min : `max(list) / min(list)`
 Lambda : `lambda args: expr`
 List Comp : `[x for x in it if cond]`
 Dict Comp : `{k:v for k,v in items()}`

25. Important Points to Remember

- ★ Indentation is very important in Python.
- ★ Python is case-sensitive.
- ★ Use meaningful variable names.
- ★ Always close files after use.
- ★ Handle exceptions to avoid crashes.
- ★ Practice coding regularly.
- ★ Read official Python documentation.
- ★ Build projects to strengthen concepts.
- ★ Keep your code clean and simple.

27. Final Tip

Consistency ★ Practice ★ Patience ★ Success in Python
 Keep learning, keep coding, and enjoy the journey!

